

Universidad de Sevilla
Máster en Ingeniería de Electrónica, Robótica y Automática (MIERA)

Control en Vehículos

Detección de Carriles y Control de Mantenimiento de Carril

Grupo 1

Agustín Mayor
Rafael Muñoz
José Francisco López

Curso 2025–2026

Índice

1. Introducción	3
2. Detección de carriles	4
3. Control de mantenimiento de carril	4
4. Integración en simulación y resultados	4
4.1. Entorno de simulación: CARLA Simulator	4
4.2. Arquitectura del sistema	5
4.3. Infraestructura: Docker y contenedores	6
4.4. Nodos ROS2 del sistema	7
4.4.1. Nodo de detección de carriles (<code>lane_detection_node</code>)	7
4.4.2. Nodo de control del vehículo (<code>vehicle_control_node</code>)	7
4.4.3. Nodo generador de anotaciones (<code>annotation_generator_node</code>)	8
4.5. Generación de datasets sintéticos	9
4.6. Resultados	10
5. Conclusiones	10

Resumen

Este trabajo presenta el diseño e implementación de un sistema de *Lane Keeping Assist* (LKA) para un vehículo autónomo simulado en *CARLA Simulator* e integrado con *ROS2 Humble* mediante contenedores Docker. El sistema consta de tres bloques principales: detección de líneas de carril mediante visión clásica con OpenCV y la transformada de Hough, diseño y sintonización de un controlador PID para mantener el vehículo centrado en el carril, y la integración de ambos módulos en un entorno de simulación realista donde se valida el funcionamiento completo. El código fuente, datasets e imágenes y vídeos demostrativos están disponibles en el repositorio público de GitHub: <https://github.com/JoseLopez36/Deteccion-Seguimiento-Carril>.

1. Introducción

Los sistemas de asistencia a la conducción (ADAS) son hoy en día un componente fundamental tanto en la industria del automóvil como en la investigación en robótica móvil. Entre estos sistemas destaca el *Lane Keeping Assist* (LKA), cuyo objetivo es mantener el vehículo centrado en su carril de forma autónoma, corrigiendo continuamente la dirección en función de la posición relativa del vehículo respecto a las marcas viales.

Este trabajo aborda el diseño e implementación de un sistema LKA completo, desde la adquisición de imagen hasta la actuación sobre los mandos del vehículo, validado en el simulador *CARLA Simulator* mediante *ROS2 Humble*. El flujo de funcionamiento del sistema es el siguiente:

1. Adquisición de imágenes desde la cámara RGB frontal del vehículo simulado.
2. Preprocesado de la imagen y detección de las líneas del carril mediante visión clásica.
3. Cálculo del error lateral del vehículo respecto al centro del carril, expresado en metros a partir de los parámetros intrínsecos de la cámara.
4. Generación de los comandos de aceleración/frenado y dirección mediante controladores PID, sintonizados previamente en MATLAB/Simulink.
5. Publicación de los comandos de control al simulador CARLA.

Todo el código fuente, datasets y material multimedia están disponibles en el repositorio público de GitHub del proyecto:

<https://github.com/JoseLopez36/Deteccion-Seguimiento-Carril>

Estructura de la memoria

La memoria se organiza en tres bloques principales:

Sección 2: Detección de carriles. Describe el pipeline de visión artificial empleado para detectar las marcas viales y calcular el error lateral del vehículo.

Sección 3: Control de mantenimiento de carril. Explica el modelado del vehículo y la sintonización del controlador PID en MATLAB/Simulink.

Sección 4: Integración en simulación y resultados. Desarrolla la arquitectura software completa en CARLA/ROS2, la infraestructura Docker, los nodos implementados y los resultados obtenidos.

2. Detección de carriles

3. Control de mantenimiento de carril

4. Integración en simulación y resultados

Esta sección describe cómo los módulos de detección de carriles y control PID se integran en un sistema completo que opera en simulación en tiempo real. Se presentan el entorno de simulación empleado, la arquitectura software basada en ROS2, la infraestructura de contenedores Docker, los nodos que componen el sistema, la herramienta de monitorización y los resultados experimentales obtenidos.

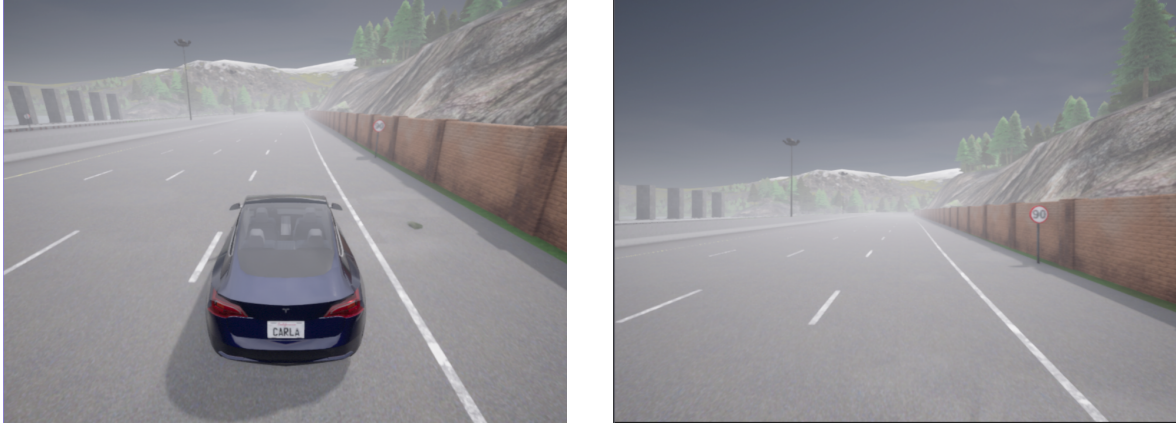
4.1. Entorno de simulación: CARLA Simulator

Para el desarrollo y validación del sistema se ha empleado *CARLA Simulator* (*Car Learning to Act*), un simulador de conducción autónoma de código abierto basado en el motor gráfico *Unreal Engine 4*. CARLA proporciona sensores virtuales de alta fidelidad —cámaras RGB, cámaras de segmentación semántica y medidores de velocidad— que permiten diseñar, depurar y validar algoritmos de percepción y control sin necesidad de un vehículo real.

El simulador se ejecuta en su versión *0.9.15* dentro de un contenedor Docker con acceso a la GPU de la máquina anfitriona. Las principales características del entorno configurado son:

- Mapa: Town04, escenario de carretera inter-urbana con marcas viales bien definidas.
- Vehículo (*ego-vehicle*): modelo basado en el Tesla Model 3, controlado en lazo cerrado desde ROS2.
- Sensores disponibles:
 - Cámara RGB frontal (**rgb_front**): imagen principal usada para la detección de carriles.
 - Cámara de segmentación semántica (**semantic_segmentation_front**): proporciona la máscara de *ground truth* utilizada en la generación del dataset sintético.
 - Medidor de velocidad (**speedometer**): velocidad actual del vehículo en m/s.
 - Cámara exterior (**rgb_view**): vista en tercera persona para supervisión visual.

La Figura 1 muestra las dos perspectivas principales empleadas durante las sesiones de conducción autónoma.



(a) Vista exterior (tercera persona).

(b) Cámara frontal del vehículo.

Figura 1: Perspectivas en CARLA Simulator.

4.2. Arquitectura del sistema

El sistema completo se organiza en tres capas que se comunican a través de una red Docker en modo *host*:

1. CARLA Simulator: simulador en su propio contenedor Docker con acceso a la GPU, que publica imágenes y telemetría como tópicos ROS2.
2. CARLA-ROS2 Bridge: puente oficial (`carla_ros_bridge`) que traduce los datos de los sensores de CARLA en mensajes ROS2 estándar y convierte los comandos de control ROS2 en actuaciones sobre el vehículo simulado.
3. Paquete de detección y control (`deteccion_seguimiento_carril`): paquete ROS2 propio que aloja los nodos de detección de carriles, control del vehículo y generación de anotaciones visuales.

El flujo de datos es el siguiente: CARLA publica las imágenes de la cámara frontal en `/carla/ego_vehicle/rgb_front/image` y la velocidad en `/carla/ego_vehicle/speedometer`. El `lane_detection_node` procesa cada *frame* y publica el error lateral en metros en `/lane_detection/lane_error` y el estado completo del carril como JSON en `/lane_detection/lane_state`. El `vehicle_control_node` consume el error lateral y la velocidad para calcular y publicar los comandos de actuación en `/carla/ego_vehicle/vehicle_control_cmd`. Por último, el `annotation_generator_node` recoge el estado de todos los nodos y genera una capa de anotaciones visuales en `/foxglove/annotations`, que Foxglove Studio superpone sobre la imagen en tiempo real.

La Figura 2 muestra el diagrama de bloques completo del sistema.

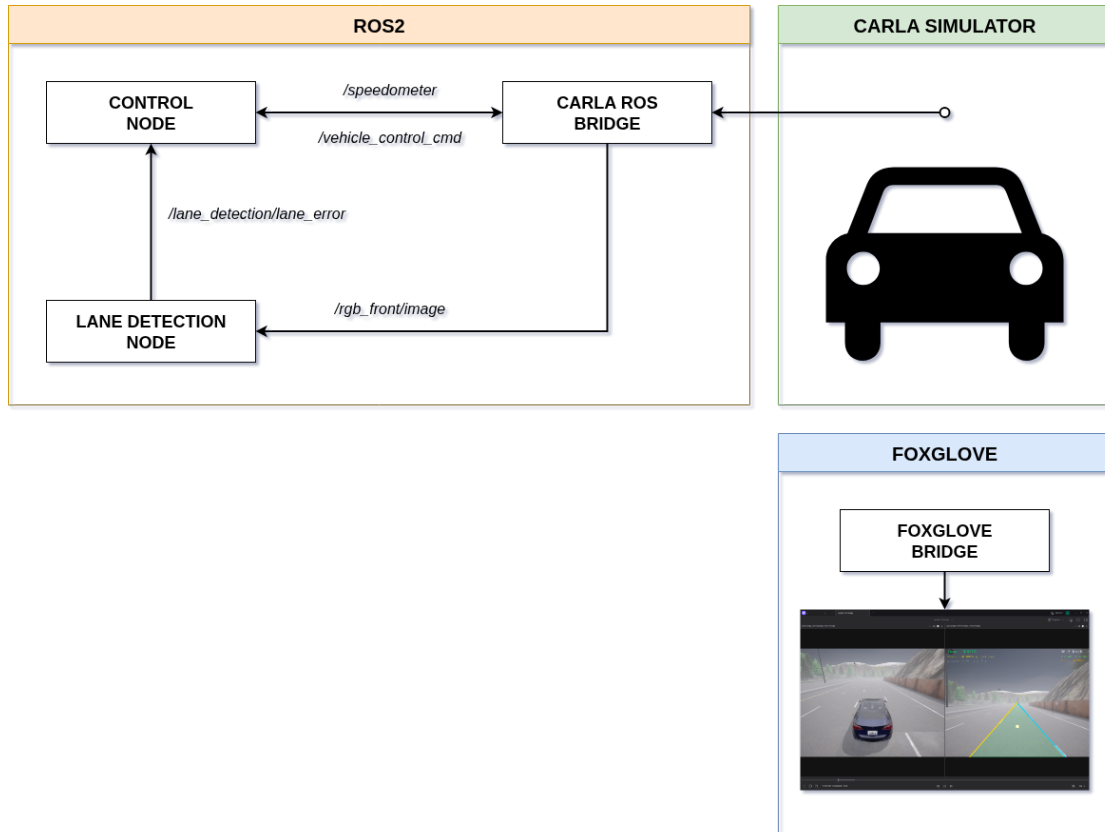


Figura 2: Diagrama de la arquitectura del sistema.

4.3. Infraestructura: Docker y contenedores

Todo el sistema se orquesta mediante **Docker Compose**, levantando los tres servicios con un único comando desde la raíz del repositorio:

```

1 xhost +local:docker
2 docker compose -f tools/docker-compose.yaml up
  
```

La siguiente tabla describe cada uno de los servicios:

Servicio	Contenedor	Función
ros_bridge	carla_ros_bridge	Bridge CARLA→ROS2
navegacion	deteccion_seguimiento_carril	Lanzamiento del paquete de ROS2
foxglove	foxglove_bridge	WebSocket bridge para Foxglove Studio

La imagen Docker base es `ros:humble-ros-base-jammy`, sobre la que se instalan ROS2 Humble Hawksbill, las librerías de visión artificial (`python3-opencv`, `ros-humble-cv-bridge`), las herramientas de visualización (`ros-humble-foxglove-bridge`, `ros-humble-foxglove-msgs`) y el cliente Python de CARLA 0.9.15 junto con el `carla-ros-bridge` compilado desde el código fuente.

El código fuente del paquete ROS2 se monta como volumen dentro del contenedor de navegación. Esto permite que cualquier cambio en el host quede disponible tras ejecutar `colcon build -symlink-install` sin necesidad de reconstruir la imagen Docker.

4.4. Nodos ROS2 del sistema

El paquete `deteccion_seguimiento_carril` implementa tres nodos ROS2 en Python que se lanzan de forma conjunta mediante `run.launch.py`, más un nodo adicional de recolecta de datos que se activa por separado.

4.4.1. Nodo de detección de carriles (`lane_detection_node`)

Este nodo recibe cada *frame* de la cámara frontal y ejecuta el pipeline de visión clásica descrito en la Sección 2. A partir de las líneas detectadas calcula el error lateral del vehículo respecto al centro del carril: primero en píxeles (`offset_px`) y luego en metros, dividiendo por la longitud focal f_x extraída del mensaje `camera_info`. Los resultados se publican en:

- `/lane_detection/lane_error` (`std_msgs/Float32`): error lateral en metros.
- `/lane_detection/lane_state` (`std_msgs/String`): JSON con las coordenadas de las líneas izquierda y derecha, la zona de posición (CENTER/LEFT/RIGHT) y los flags de detección.

4.4.2. Nodo de control del vehículo (`vehicle_control_node`)

Este nodo cierra el lazo de control: recibe el error lateral y la velocidad actual, y genera los comandos de actuación sobre el vehículo. Opera a 20 Hz e implementa dos controladores PID independientes.

PID de velocidad (control de crucero). Regula la velocidad del vehículo en torno a la referencia configurable `target_speed`. El error de control es la diferencia entre la velocidad objetivo y la medida por el velocímetro. La salida del PID se mapea a los canales *throttle* (acelerador) y *brake* (freno) normalizados en $[0, 1]$:

$$\begin{aligned}
 e_v(t) &= v_{\text{ref}} - v_{\text{actual}} \\
 u_v(t) &= K_{p,v} e_v + K_{i,v} \int e_v dt \\
 \text{throttle} &= \max(0, \min(1, u_v)), \quad \text{brake} = \max(0, \min(1, -u_v))
 \end{aligned}$$

PID de dirección (mantenimiento de carril). Ajusta el ángulo del volante para mantener el vehículo centrado en el carril. El PID opera en radianes y su salida se normaliza por el ángulo físico máximo del volante ($\theta_{\text{máx}} = 0,6 \text{ rad} \approx 34^\circ$) para obtener el valor de dirección normalizado de CARLA en $[-1, 1]$. La convención de signo es: error positivo (vehículo desplazado a la derecha) produce una corrección hacia la izquierda ($\text{steer} < 0$):

$$u_s(t) = K_{p,s} e_s + K_{i,s} \int e_s dt + K_{d,s} \dot{e}_s$$

$$\text{steer} = \max\left(-1, \min\left(1, \frac{u_s}{\theta_{\text{máx}}}\right)\right)$$

Ambos PIDs incorporan *anti-windup* que limita el término integral al rango $[-10, 10]$. Los parámetros definitivos configurados en `params.yaml` se recogen en la Tabla ???. Estos valores son el resultado de la sintonización en MATLAB/Simulink descrita en la Sección 3, trasladados directamente a la implementación en Python.

4.4.3. Nodo generador de anotaciones (`annotation_generator_node`)

Este nodo actúa como capa de visualización del sistema. En cada *frame* de la cámara frontal recoge el estado de los demás nodos y genera un mensaje `foxglove_msgs/ImageAnnotations` que Foxglove Studio superpone sobre la imagen en tiempo real. Las anotaciones incluyen:

- Polígono de carril: región semitransparente en verde que cubre el área delimitada por las líneas izquierda y derecha detectadas.
- Líneas de carril: línea izquierda en amarillo y derecha en cian, con grosor de 5 px.
- Vector de desviación: segmento desde el centro geométrico de la imagen hasta el centro estimado del carril, en amarillo.
- HUD de telemetría: zona del carril (`CENTER/LEFT/RIGHT`), error lateral en metros y en píxeles, velocidad actual en km/h, y valores de *throttle*, *brake* y *steer*.

La Figura 3 muestra la interfaz de Foxglove Studio durante una sesión de conducción.



Figura 3: Interfaz de Foxglove Studio durante una sesión de conducción autónoma. A la izquierda, la vista exterior del vehículo; a la derecha, la cámara frontal con las anotaciones superpuestas: líneas del carril, polígono de área y HUD de telemetría.

4.5. Generación de datasets sintéticos

Una de las ventajas de trabajar con un simulador como CARLA es la posibilidad de generar datos etiquetados de forma automática. La cámara de segmentación semántica asigna a cada píxel una etiqueta de clase con precisión perfecta, ofreciendo un *ground truth* ideal para evaluar el detector de carriles sin ningún coste de anotación manual.

El nodo `lane_dataset_node` opera en modo conducción manual (sin el nodo de control) y captura imágenes a 5 Hz. Para cada instante capturado:

1. Obtiene la máscara semántica del mismo instante de tiempo.
2. Detecta los píxeles de marcas viales en la máscara y los proyecta sobre la imagen RGB.
3. Almacena el par imagen RGB / máscara binaria en `dataset/lanes/images/` y `dataset/lanes/masks/` respectivamente.

La recolecta se lanza con:

```
1 xhost +local:docker
2 docker compose -f tools/docker-compose-lane-dataset.yaml up
```

El repositorio incluye además la herramienta `tools/visualize_lanes_dataset.py` para revisar el dataset generado y `tools/make_lane_video.py` para producir un vídeo con el detector de carriles aplicado sobre las imágenes del dataset.

4.6. Resultados

Las pruebas de conducción autónoma se han realizado a dos velocidades de referencia: 18 km/h (5.0 m/s) y 30 km/h (8.33 m/s), modificando únicamente el parámetro `target_speed` en `params.yaml`. En ambos casos el sistema es capaz de mantener el vehículo centrado en el carril de forma continuada sin intervención humana.

Los vídeos de las sesiones de conducción autónoma a 18 km/h y 30 km/h, así como un vídeo de demostración del detector de carriles sobre el dataset sintético, están disponibles en el repositorio público de GitHub:

<https://github.com/JoseLopez36/Deteccion-Seguimiento-Carril>

Los ficheros correspondientes son `media/Demo_18_kmh.mp4`, `media/Demo_30_kmh.mp4` y `media/Deteccion_Carriles.mp4`.

5. Conclusiones

Este trabajo ha permitido implementar un sistema de *Lane Keeping Assist* funcional de extremo a extremo, demostrando que la combinación de técnicas de visión clásica con un controlador PID en cascada y una arquitectura ROS2 modular constituye una solución eficaz y fácilmente reproducible para la conducción autónoma en simulación.